Qwen

Beyond Black Boxes: A Modular, Vectorized-Event Driven Architecture for Validating the P3 Candlestick Reversal Strategy

Comparative Analysis of Free/Open-Source Backtesting Platforms

The selection of a backtesting platform for the Protocol 3 (P3) Candlestick Reversal v3.0 strategy necessitates a departure from traditional, integrated backtesting engines. The user's directive mandates a modular architecture that prioritizes debugging transparency through explicit, per-filter boolean masking over the convenience of a monolithic framework [3]. This requirement fundamentally reshapes the evaluation criteria, moving away from a simple feature-comparison matrix towards an analysis of architectural flexibility, composability, and the fidelity of its underlying data structures. The core challenge lies in finding a stack where the signal generation logic can be executed in a massively parallel, vectorized fashion, while trade simulation occurs in a granular, event-by-event manner, all within a single, coherent workflow.

A primary obstacle identified in initial attempts with `backtesting.py` is its struggle to scale with the complexity of multi-candle pattern logic and instrument-specific parameter routing [5]. While it supports basic vectorization through `.shift()` operations, constructing the intricate 8-filter convergence stack becomes cumbersome and opaque, obscuring which specific gate rejects a potential trade. This lack of transparency is a critical failure for a calibration-focused strategy. Similarly, its default execution model assumes ideal conditions—fills at the next bar's open—which is fundamentally incompatible with the realistic friction modeling required for Over-the-Counter (OTC) markets like Forex and CFDs on commodities and indices [4]. The degradation in performance metrics observed when transitioning from vectorized to event-driven models underscores the necessity of simulating real-world mechanics like bid-ask spreads, slippage, and market impact [4].

Consequently, the analysis of existing platforms reveals that no single tool perfectly meets the user's non-negotiable requirements. Instead, the optimal solution involves architecting a composite stack that leverages the specialized strengths of several open-source libraries. This approach allows for a clear separation of concerns: a powerful vectorized engine for generating signals across all 35 instruments, followed by a dedicated event-driven component for simulating realistic trade execution. This hybrid model directly addresses the shortcomings of `backtesting.py` by providing the debugging clarity of vectorization and the execution realism of an event-driven loop [1] [4].

The table below presents a comparative analysis of leading free and open-source Python-based backtesting frameworks against the specific technical constraints of the P3 strategy. The scoring reflects each tool's suitability for the mandated modular, hybrid architecture rather than its utility as a standalone solution. Key differentiators include the ability to process multi-index DataFrames for batch instrument analysis, the availability of declarative expression engines for complex rule-based logic, and the flexibility to integrate custom, event-driven execution logic.

Feature / Tool	`vectorbt`	`backtrader`	`backtesting.py`	`Qlib`	`Lean/ QuantConnect (local)`
Multi-Instrument Batching	Excellent (native support for nested arrays/ DataFrames) [5]	Good (requires manual DataFrame concatenation or looping) [5]	Fair (single DataFrame per backtest run)	Excellent (via `DatasetProvider` with multi-index DataFrame) [3]	Good (designed for multiple securities)
Vectorization vs. Event-Driven	Primarily vectorized	Primarily event-driven	Primarily vectorized	Primarily vectorized	Primarily event-driven
Pattern Logic Implementation Ease	High (powerful array manipulation)	Low (logic must be written inside event loop)	Medium (cumbersome with `.shift()`)	Very High (declarative expressions via `ExpressionProvider`) [3]	Low (requires custom indicator classes)
Filter Stack Transparency	Excellent (outputs boolean arrays)	Poor (obscured within event loop)	Poor (final signal only)	Excellent (filters applied via expression engine) [3]	Poor (obscured within engine)
Cost Modeling Flexibility	Moderate (customizable)	Excellent (highly customizable broker/sizer) [1]	Low (limited to fixed fees/spread)	Moderate (configurable)	Excellent (highly configurable)
Learning Curve	Moderate	High	Low	High	High
Overall Recommendation	Core of the Vectorized Engine	Core of the Event-Driven Simulator	Not Recommended	Potential Signal Engine Alternative	Not Recommended

Based on this analysis, the most pragmatic and robust architecture is not a single library but a composite stack built upon the Python ecosystem. The recommended approach is to construct a custom pipeline where `pandas` serves as the foundational data structure. For the signal generation phase, this involves leveraging `numpy`'s capabilities alongside `pandas` for efficient, batch processing of all 35 instruments simultaneously. This vectorized engine will compute all six candlestick patterns and the eight filters, producing a comprehensive set of intermediate boolean masks. This step directly fulfills the critical need for debugging transparency.

For the second phase, the event-driven trade simulation, `backtrader` emerges as a strong candidate due to its mature, well-documented event-driven architecture [1]. Its design, which decouples signal logic from execution, aligns perfectly with the hybrid model. By feeding the final converged signal DataFrame into a `backtrader`-based engine, one can simulate fills based on ask/bid prices, apply realistic slippage models, manage position sizes dynamically, and execute complex order types like scale-outs at predefined risk multiples (+1R, +2R). This two-stage process elegantly combines the speed and analytical power of vectorization with the fidelity and realism of event-driven simulation, forming the cornerstone of a production-ready validation framework for the P3 strategy.

Modular Data Pipeline Architecture and Implementation

The integrity of any backtest hinges on the quality, consistency, and timeliness of its underlying data. For the P3 strategy, which spans six years of H4 and D1 data across 35 diverse instruments, a robust, automated, and DST-aware data pipeline is not merely a supporting component but the bedrock of the entire architecture. The pipeline must be designed to ingest, clean, and structure heterogeneous data sources—including OHLCV, volatility indices, economic calendar events, and liquidity proxies—into a unified format suitable for both vectorized signal generation and event-driven simulation. The recommended storage format is Apache Parquet, chosen for its high compression ratio, fast read/write speeds, and schema enforcement capabilities, which ensure data consistency across runs [5].

The first step is sourcing reliable and free data. For historical OHLCV data, several options exist. The `yfinance` Python library is a popular starting point for downloading daily data for major indices and forex pairs, though its coverage for exotic crosses and

commodities may be limited. For more comprehensive historical data, providers like Alpha Vantage or the free tier of Polygon.io offer extensive datasets, often available via API. Given the static nature of the project, a one-time download and caching of this data is sufficient. The key is to ensure all incoming timestamps are standardized to UTC, which is the international standard for financial data indexing and prevents ambiguity [4]. This normalization is a prerequisite for all subsequent timezone conversions.

Next, ancillary data streams must be integrated. Volatility regime tagging requires data for the VIX, GVZ (VIX-Gold correlation index), and OVX (Oil volatility index). These can typically be found on financial data portals like Yahoo Finance or Quandl. Economic calendar events (EIA, API, FOMC, NFP) are critical for implementing news blackout filters. Publicly available CSV files from websites like Forex Factory or Investing.com can be parsed to extract event names and scheduled release times. Liquidity proxies for OTC markets, which lack centralized volume data, are challenging to source. A practical approach is to use tick volume data if available from a broker, or proxy it using the Average True Range (ATR). An increase in ATR often correlates with higher liquidity and volatility. This ATR(14) value will be crucial for both the volatility filter and the slippage model.

Once data is acquired, a systematic cleaning and structuring process is essential. Gaps in the historical data, particularly around holidays or due to reporting delays, must be handled. The recommended method is forward-filling only the closing price (`Close`). All other fields (`Open`, `High`, `Low`) should be dropped if a bar is missing, as they cannot be reliably interpolated without introducing lookahead bias. Timezone handling is another critical aspect. Timestamps must be converted from their local market time to UTC during ingestion. When performing session filtering later, the UTC timestamp must be converted back to the relevant local market time (London, New York, Tokyo) using a robust library like `pytz` or `dateutil`. This ensures that session windows, such as London 08:00–16:00 GMT, remain consistent and do not break during Daylight Saving Time (DST) transitions in March and October [3]. The following pseudo-code illustrates this process:

```
import pandas as pd
import pytz

# Assume 'df' has a UTC-timestamped index
def convert_to_local_tz_and_filter(df, market='London'):
    tz_map = {'London': pytz.timezone('Europe/London'),
              'NewYork': pytz.timezone('America/New_York'),
              'Tokyo': pytz.timezone('Asia/Tokyo')}
    local_tz = tz_map[market]
```

```
    # Convert UTC index to local timezone
    df_local = df.tz_localize('UTC').tz_convert(local_tz)

    # Extract hour and minute for filtering
    hour = df_local.index.hour
    minute = df_local.index.minute

    # Define session windows (example for London)
    is_london_session = (hour >= 8) & (hour < 16)
    return df[is_london_session]
```

Finally, all cleaned datasets must be merged into a master DataFrame. Instrument-specific metadata, such as asset class and calibration factors (e.g., Gold's 1.5x ATR multiplier), should be stored in a separate DataFrame and joined with the main data frame. This allows for dynamic parameter routing throughout the signal generation process. After merging and alignment, the final, comprehensive DataFrame should be saved in Parquet format for efficient I/O. This structured, modular pipeline ensures that the data feeding the backtesting engine is accurate, consistent, and ready for the demanding computational tasks ahead.

Hybrid Signal Generation and Event-Driven Simulation Code Structure

The proposed architecture for the P3 strategy is a hybrid model that intelligently separates the computationally intensive task of signal generation from the stateful, sequential nature of trade simulation. This design directly addresses the documented limitations of purely vectorized systems like `backtesting.py`, which fail to provide the necessary debugging transparency and realistic execution fidelity required for this complex strategy [3] [4]. The architecture consists of two distinct, yet seamlessly integrated, modules: a fully vectorized signal engine for pattern and filter logic, and an event-driven portfolio simulator for trade execution mechanics. Both modules operate on a unified pandas DataFrame, ensuring a smooth data flow and maximum efficiency.

The **Vectorized Signal Engine** forms the analytical heart of the framework. Its purpose is to process the entire historical dataset (35 instruments x 6 years x H4 bars) in a single, batch operation, leveraging the parallel processing power of `numpy` and `pandas`. This

engine will iterate through the data once, calculating all six candlestick patterns and applying the eight filters sequentially, producing a series of intermediate boolean masks. This output is the cornerstone of the debugging capability, allowing developers to isolate exactly which condition fails for any given signal. The core logic for pattern detection will rely heavily on `pandas` methods like `.shift()`, `.rolling()`, and boolean indexing. For instance, detecting a Hammer pattern would involve creating boolean conditions based on the relationships between the current bar's `Close`, `Open`, `High`, and `Low` and the previous bar's `Close`.

Here is a conceptual implementation snippet for a Hammer pattern detection, assuming a DataFrame `df` with standard OHLC columns:

```
# Calculate body and wick lengths for current and prior candles
df['body_current'] = abs(df['close'] - df['open'])
df['body_prior'] = abs(df['close'].shift(1) - df['open'].shift(1))
df['upper_wick_current'] = df['high'] - df.max(axis=1)
df['lower_wick_current'] = df.min(axis=1) - df['low']

# Hammer Pattern Conditions (Bullish Reversal)
condition1 = df['lower_wick_current'] >= (2 * df['body_current'])  # Lower
condition2 = df['upper_wick_current'] <= (0.25 * df['body_current']) # Upp
condition3 = df['close'] >= (df['low'] + (df['high'] - df['low']) * (2/3))

df['mask_bullish_hammer'] = condition1 & condition2 & condition3
```

This process is repeated for all six patterns and eight filters, with each resulting mask being a column in the final signal DataFrame. This approach is vastly superior to attempting the same logic within a nested loop or an event-driven framework's `next()` method, as it is computationally efficient and inherently transparent [5].

Upon completion of the signal engine, the resulting DataFrame, containing columns for each pattern, filter, and the final converged signal (`Signal_Bull`, `Signal_Bear`), is passed to the **Event-Driven Portfolio Simulator**. This module is responsible for translating the abstract signals into concrete trades, complete with realistic costs and execution dynamics. It will consume the signal DataFrame bar-by-bar, maintaining the state of the portfolio (cash, positions, open orders). Drawing inspiration from the `backtrader` framework, this simulator will implement an event loop that calls `buy()` or `sell()` methods based on the signals [1]. However, unlike `backtesting.py`, the fill logic will be highly customized. Fill prices will be simulated based on the ask price for buys and the bid price for sells, sourced from the `high` and `low` columns of the OHLCV

data. Slippage will be modeled as a random variable, perhaps drawn from a normal distribution centered on a multiple of the current spread or ATR(14). Position sizing will be managed based on a fixed percentage of capital or a dynamic risk model. Crucially, this layer will also handle the strategy's complex exit logic, including scale-out orders at +1R and +2R targets, which requires tracking the entry price and unrealized PnL for each position. This event-driven approach captures the microstructure of the market, providing a far more realistic assessment of performance than a simple vectorized calculation [4]. The combination of these two modules creates a powerful, flexible, and transparent system perfectly tailored to the demands of the P3 strategy.

Statistical Validation and Performance Stratification Framework

A backtest is only as credible as its statistical validation. For the P3 strategy, achieving statistical rigor is paramount to distinguishing genuine alpha from random noise, especially given the complex, multi-parameter nature of the 8-filter stack. The validation framework must go beyond simple metrics like Net Profit or Max Drawdown and instead focus on establishing confidence in the results, identifying profitable sub-strategies, and mitigating the risk of overfitting. This involves enforcing minimum sample size thresholds, calculating robust confidence intervals, stratifying performance by market regimes, and implementing walk-forward validation.

The first principle of validation is sample size control. The user directive specifies that any result group with fewer than 30 trades (`N < 30`) should be flagged as having `LOW CONFIDENCE`. This is a conservative but prudent approach, as small sample sizes are highly susceptible to outliers and statistical flukes. For groups with a larger sample size (where `N` is at least 30), the framework must calculate the 95% Confidence Interval (CI) for the win rate using the Wilson Score formula. The Wilson Score CI is preferred over the standard Wald interval because it performs better for small samples and proportions near 0 or 1, providing a more accurate measure of uncertainty. The formula for the Wilson Score CI is as follows:

$$\text{Wilson Center} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

where $\hat{p}$ is the observed win rate, n is the number of trades, and z is the z-score (1.96 for 95

This calculation will be performed for every instrument, pattern, and regime combination, providing a nuanced view of where the strategy's edge is statistically significant.

The second pillar of validation is performance stratification. The strategy's effectiveness likely varies depending on the prevailing market environment. To capture this, the backtest results must be stratified by volatility and trend regimes, using the VIX and ADX(14) as indicators. The VIX can be used to create three buckets:

- **Low Volatility:** VIX < 16
- **Normal Volatility:** $16 \leqslant$ VIX $\leqslant 25$
- **High Volatility:** VIX > 25

Similarly, the ADX(14) can be used to gauge trend strength:

- **Weak Trend/Range:** ADX < 25
- **Strong Trend:** ADX $\geqslant 25$ (aligned with the strategy's Regime Gate)

By cross-referencing these two sets of buckets, four distinct market environments can be created. The strategy's performance (win rate, expectancy, drawdown) will then be calculated for each of these eight unique combinations (e.g., Win Rate @ Low Volatility / Strong Trend). This analysis can reveal whether the strategy excels in trending markets, performs well during calm periods, or falters under high stress, providing invaluable insights for risk management and future development.

Third, the framework must correctly calculate the blended R-multiple expectancy for strategies involving partial exits, such as the P3 strategy's 50% scale-out at +1R. The overall expectancy is not a simple average of the outcomes. Instead, it must account for the probability of each outcome. If a trade has a 60% chance of hitting the +1R target (with 50% of the position closed) and a 40% chance of hitting the stop-loss (-1R, with the remaining 50% position), the expected value is calculated as:

$$\text{Expectancy} = (P_{+1R} \times +1R) + (P_{Stop} \times -1R)$$

In this example, the expectancy would be `(0.6 \times +1) + (0.4 \times -1) = +0.2R`. This weighted calculation provides a true measure of the strategy's long-term profitability.

Finally, to combat overfitting, the framework must incorporate walk-forward optimization and out-of-sample testing. This involves dividing the total backtest period (2020–2026) into multiple segments. In each segment, a portion is used for "training" or parameter optimization, and the subsequent, unseen portion is used for "testing." The parameters optimized in one training window are then carried forward to the next. This process mimics how a strategy would be deployed in real-time and provides a much more robust estimate of its future performance than a single, static backtest. By combining these rigorous validation techniques, the framework moves beyond simple performance reporting to become a powerful tool for scientific discovery and risk assessment.

Phased Implementation Roadmap and System Deployment

A successful implementation of the P3 strategy's backtesting framework requires a structured, iterative approach. Attempting to build and validate a system for 35 instruments and a complex 8-filter stack from day one is a recipe for failure. A phased roadmap allows for early feedback, risk mitigation, and the progressive accumulation of confidence in the system's accuracy before committing significant computational resources. This plan outlines a three-phase deployment, detailing milestones, validation gates, and resource requirements for each stage.

Phase 1: Core Logic Validation (Timeline: 2-4 weeks) The objective of Phase 1 is to establish the fundamental correctness of the signal generation and simulation logic on a minimal viable set of instruments. This phase focuses on building the core components and verifying them against known historical events or manually verified setups.

- **Milestones:**

```
1.  <strong>Data Pipeline Completion:</strong> Source and clean H4/D1 hist
2.  <strong>Vectorized Signal Engine MVP:</strong> Implement the vectorize
3.  <strong>Event-Driven Simulator MVP:</strong> Build a basic event-drive
4.  <strong>Backtest Execution and Debugging:</strong> Run the backtest on
```

- **Validation Gates:**
- The output must show a clear correspondence between visual chart patterns and the generated signals/masks.

- The final net PnL from the event-driven simulator must be demonstrably lower than a naive vectorized calculation (which ignores friction), confirming that the simulator is correctly applying costs.
- All unit tests for individual pattern and filter logic must pass.
- **Compute Requirements:** Standard laptop or desktop (8GB RAM, modern CPU). No GPU acceleration is needed at this stage.
- **Version Control:** Establish a Git repository. Each major milestone should correspond to a tagged release (e.g., `v0.1-core-engine-built`).

Phase 2: Parameter Routing and Scaling (Timeline: 3-5 weeks) With the core logic validated, Phase 2 expands the system's capacity. The focus shifts to implementing the instrument-specific parameter routing and increasing the instrument count to 23.

- **Milestones:**

```
1.  <strong>Dynamic Parameter Integration:</strong> Create an instrument m
2.  <strong>Session and News Filter Implementation:</strong> Integrate the
3.  <strong>Advanced Cost Modeling:</strong> Enhance the event-driven simu
4.  <strong>Expanded Backtest:</strong> Re-run the backtest on the expande
```

- **Validation Gates:**
- The strategy's performance must change meaningfully when instrument-specific parameters are enabled versus a uniform setting, demonstrating that the routing logic is working.
- The number of signals and overall performance should decrease after adding the session and news filters, as intended.
- The introduction of realistic cost modeling should cause a further reduction in net PnL, reflecting the true cost of trading.
- **Compute Requirements:** Same as Phase 1. The increased number of instruments will raise memory usage but should still be manageable on a standard machine.
- **Version Control:** Continue using Git. Feature branches should be used for developing new functionality (e.g., `feature/session-filter`) before merging into the main branch.

Phase 3: Full-Scale Validation and Statistical Analysis (Timeline: 4-6 weeks) Phase 3 represents the culmination of the project. The full 35-instrument system is tested, and the statistical validation framework is applied to derive meaningful conclusions about the strategy's performance.

- **Milestones:**

```
1.  <strong>Full Instrument Backtest:</strong> Execute the complete backte
2.  <strong>Statistical Analysis:</strong> Apply the full suite of statist
3.  <strong>Walk-Forward Validation:</strong> Implement a walk-forward ana
4.  <strong>Reporting:</strong> Generate a comprehensive report summarizin
```

- **Validation Gates:**
- The final report must clearly delineate between statistically significant results and those that are insignificant due to low sample size or poor performance.
- The walk-forward analysis should show stable performance across different market regimes, indicating that the strategy is not overfitted to a specific historical period.
- The entire process, from data ingestion to final reporting, must be reproducible, with all scripts and configurations version-controlled.
- **Compute Requirements:** A high-performance machine or cloud virtual machine (e.g., AWS EC2 r5.large or equivalent) with 32GB RAM and a multi-core processor is recommended to handle the large dataset efficiently.

This phased approach de-risks the project by breaking it down into manageable chunks, ensuring that each component is thoroughly vetted before moving on to the next, ultimately leading to a robust and reliable backtesting framework.

Critical Pitfalls and Anti-Patterns in Strategy Backtesting

Developing a sophisticated backtesting framework for a strategy as complex as P3 exposes numerous opportunities for error. Avoiding common pitfalls and anti-patterns is as important as building the core logic itself. These issues often introduce subtle biases that can invalidate an entire backtest, leading to false confidence in a flawed strategy. The following sections detail the most critical anti-patterns to avoid, focusing on data handling, logical errors, and the misinterpretation of results.

The most insidious anti-pattern is **Look-Ahead Bias**, where information from the future is inadvertently used to make a decision at the present. This can occur in several ways. The most common is failing to use `.shift()` correctly in vectorized code. For example, in a pattern detection function, using `df['close'].shift(1)` is correct for referencing the previous close, but using `df['close'].shift(-1)` would be catastrophic, as it introduces information from the next bar. Another form of look-ahead occurs during data preprocessing. Forward-filling gaps in a time series is acceptable, but doing so naively can

be problematic. The rule must be to only forward-fill the `Close` price. Forward-filling `Open`, `High`, or `Low` values is a severe violation of causality, as these prices are not known at the start of the bar. Furthermore, when resampling data from a lower frequency (e.g., 1-hour) to a higher frequency (e.g., 4-hour), care must be taken to ensure that the 4-hour bar's data is constructed exclusively from data points within its own time window, and not from the next bar's data.

A second major pitfall is **Survivorship Bias**, which occurs when a backtest is run only on assets that have survived until the present day, excluding those that have gone bankrupt or been delisted. While less of an issue for perpetual instruments like major currency pairs, it is a critical consideration when including equities or ETFs. The historical data feed must include delisted symbols, or a comprehensive survivorship bias correction must be applied to the data, otherwise the strategy will appear artificially profitable by having been tested only on the "winners."

Third, **Over-Optimization (Curve-Fitting)** is a pervasive danger. The P3 strategy's 8-filter stack and instrument-specific calibrations represent many degrees of freedom. Optimizing each parameter to maximize a metric like Sharpe Ratio on a static backtest period is a classic path to curve-fitting. The results will almost certainly not hold up in out-of-sample data or live trading. The recommended mitigation is the use of **walk-forward analysis**, as detailed in the roadmap. This technique forces the model to adapt to changing market conditions and provides a much more realistic estimate of future performance. Additionally, the principle of Occam's Razor should be applied: simpler strategies with fewer parameters are generally more robust and less prone to overfitting.

Fourth, **Unrealistic Friction Modeling** leads to inflated and misleading performance figures. As established, assuming fills at the mid-price or next bar's open is a significant error for OTC markets [4]. The anti-pattern here is to underestimate transaction costs. Simply adding a fixed pip or dollar amount for slippage is often insufficient. A more robust model, as recommended for the P3 framework, is to base slippage on a variable proxy like the Average True Range (ATR) or the current bid-ask spread. This makes the slippage cost stochastic and dependent on market conditions, reflecting reality more accurately. Similarly, swap/rollover costs for CFDs must be included, as they can significantly erode profits over longer holding periods.

Finally, there is the anti-pattern of **Misinterpreting Statistical Significance**. A positive net PnL is not enough. The framework must enforce minimum sample size thresholds (`N >= 30`) and use appropriate statistical tests. Calculating the Wilson Score Confidence Interval for win rates in small sample groups is a direct countermeasure to this pitfall. Presenting a win rate of 60% for a strategy with only 5 trades is meaningless. Flagging

such results as `LOW CONFIDENCE` is a critical safeguard against drawing erroneous conclusions. Furthermore, simply observing a positive expectancy in one market regime (e.g., high volatility) is insufficient. The strategy must demonstrate robustness across all defined regimes (low, normal, high volatility; weak trend, strong trend) to be considered truly diversified and resilient. By actively identifying and guarding against these anti-patterns, the development process ensures that the final backtest results are not just impressive on paper, but represent a genuine, statistically sound edge.

Reference

1. Can someone help me with the code? It's not working properly https://stackoverflow.com/questions/76586842/can-someone-help-me-with-the-code-its-not-working-properly
2. Building Trading Bots and Backend Systems | Favour Olusayo ... https://www.linkedin.com/posts/favour-olusayo_programming-tradingsystems-mql5-activity-7333627427572633600-OtX8
3. API Reference — QLib 0.9.8.dev11 documentation https://qlib.readthedocs.io/en/latest/reference/api.html
4. Vectorized Backtests Lie: Event-Driven Backtesting for Real Results https://www.linkedin.com/posts/michael-fernandes-7a3b6227a_backtest-vs-reality-activity-7408478160607932416-BMgK
5. Vectorized Backtester in Pandas/Python: Loop through each stock ... https://stackoverflow.com/questions/68191080/vectorized-backtester-in-pandas-python-loop-through-each-stock-as-a-new-datafra